SCHOOL OF NATURAL AND APPLIED SCIENCES

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY (CSIT)

PROJECT - NPRJ740

ADVANCED ICT DIPLOMA - 2026

SUPERVISOR: DR TITE TUYIKEZE

PHASE 2

System Requirements, Modelling and Project Management

Field	Details
Name of Document	Phase 2: System Requirements, Modelling and Project Management
Project Title	AgroAdvisor: an AI-assisted farm advisory platform for smallholder farmers
Name of the System	AgroAdvisor
Student Name	[Student name]
Student Number	[Student number]
Submission Date	[Confirm submission date with the supervisor]

Yellow-highlighted text marks content that still needs a personal detail or real evidence from the student.

TABLE OF CONTENTS

1. System Functional and Non-Functional Requirements 3
1.1 Product features (functional requirements) 3
1.2 System non-functional requirements 4
2. System Modelling 4
2.1 Context diagram 4
2.2 Architectural design 5
2.3 System design models 6
3. Project Management 10
3.1 Work breakdown 10
3.2 Gantt chart 13
4. Conclusion 13

1. System Functional and Non-Functional Requirements

AgroAdvisor is built for one type of user: the smallholder farmer. There are no administrator, buyer or other roles. The system interprets weather data together with each farmer's farm profile (location, crops, livestock) to give personal advice and severe-weather warnings, instead of only showing the weather.

Priority uses the MoSCoW scale: M = Must have, S = Should have, C = Could have. Status shows what exists in the current version of the system.

1.1 Product features (functional requirements)

ID	Requirement	Priority	Status
FR1	A farmer can register with full name, email and password, and can sign in with email and password.	M	Built
FR2	A farmer can sign in with a Google account. A Microsoft sign-in button exists but the provider is not yet enabled in Firebase.	S	Google built; Microsoft pending
FR3	A farmer can create and edit one farm profile: name, nearest town, location on a map, size (acres or hectares), crops and livestock with counts, and an optional photo.	M	Built
FR4	A farmer can set the farm location by tapping a map or using the device location.	M	Built
FR5	A farmer can estimate plot size by walking the boundary with the device GPS. GPS points are not saved.	C	Built
FR6	A farmer can add a crop or animal by taking or choosing a photo, which the system identifies, or by typing its name.	S	Built
FR7	The dashboard shows current weather for the farm (temperature, conditions, high and low, humidity, feels-like) next to the farm name.	M	Built
FR8	The system shows a 7-day farm outlook with an interpretation and a risk note for each day, based on the farm's crops and livestock.	M	Built
FR9	The system shows a severe-weather warning (heat, frost, heavy rain, strong wind, storm) with what to do, when and why, and can read the warning aloud.	M	Built
FR10	The farm profile map shades nearby estimated temperatures (cool, mild, warm, hot).	C	Built
FR11	A farmer can browse reviewed and published video lessons grouped by topic, with short key steps.	M	Built
FR12	A farmer can mark a lesson complete. The completion is saved in the farm memory.	S	Built
FR13	A farmer can record a weather event, planting or what worked, and see a weekly summary and recent entries. Entries can be removed.	S	Built
FR14	A farmer can add a local observation (flooding, frost or other) within 25 km of their farm and see visible notes nearby on a map.	S	Built
FR15	A farmer can report another farmer's note. A note is hidden after two different farmers report it. The author can remove their own note.	S	Built
FR16	A farmer can change the interface language (English, Afrikaans, isiZulu, Sesotho, Setswana).	S	Built
FR17	The system sends a background push notification when severe weather is forecast, even if the app is closed.	C	Not built: needs a service-account key

1.2 System non-functional requirements

The targets below are design targets. They have not yet been measured in a formal test, so no measured result is claimed.

ID	Quality	Requirement (design target)
NFR1	Reliability	If the weather service or the Worker is unavailable, the dashboard shows a clear error with a retry option and does not show wrong data. Saved farm data is never changed by a failed forecast request.
NFR2	Availability	The client (Cloudflare Pages), the Worker (Cloudflare) and the database (Firebase) are managed cloud services, so no server needs to be maintained by the farmer or the developer. Availability is therefore that of these providers.
NFR3	Security	Every farmer can read and write only their own farmer, farm and memory data. Firestore Security Rules enforce this on the server. The Worker verifies the Firebase ID token on each request. Secrets are kept in environment files and Wrangler secrets, never in source code.
NFR4	Security	Community notes contain no names. Direct database writes are limited by rules: note length, 25 km radius, one report per farmer per note, and report counts that cannot be forged. Six automated rule tests cover these cases.
NFR5	Maintainability	TypeScript strict mode with no 'any'. Firestore access only through service modules. All agronomic thresholds live in one knowledge-base file with a comment explaining each rule. The worker uses one shared response helper.
NFR6	Portability	The client is a web application that runs in any modern browser on phone or computer. No app-store installation is needed.
NFR7	Performance	The interface is mobile-first and the build is split into smaller files so the first screen loads quickly on slow mobile data. The map is loaded only when needed. Target: the dashboard shows weather soon after the forecast response arrives. [Measure and record load times before submission.]
NFR8	Usability	Plain-language advice, large touch targets (buttons at least 44 px high), labelled form controls, sufficient colour contrast and a loading, empty and error state on every data screen.
NFR9	Cost	The system stays on free plans: Firebase Spark, Cloudflare free tier and Open-Meteo. No Cloud Functions are used.

2. System Modelling

2.1 Context diagram

The context diagram shows AgroAdvisor as one process, the farmer as the only user, and the external systems it exchanges data with.

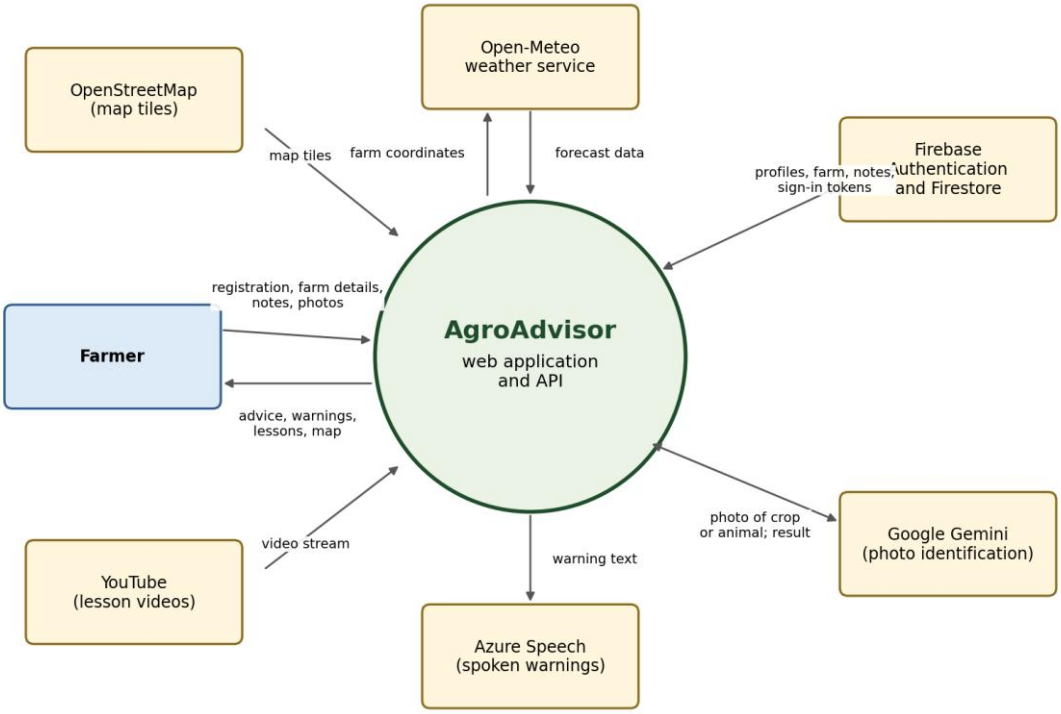


Figure 1: Context diagram of AgroAdvisor

2.2 Architectural design

AgroAdvisor uses a client-server model organised in three layers.

Layer	Components	Responsibility
Presentation	React + TypeScript + Tailwind client on Cloudflare Pages: pages, components, service modules, weather-advice rules	Shows screens, collects input, reads and writes the farmer's own Firestore data through service modules, and turns forecast data into advice and warnings using the knowledge-base thresholds.
Application	Hono API on a Cloudflare Worker: authentication middleware, routes under /api (health, weather, vision, speech), forecast and regional-grid services	Verifies the Firebase ID token with the jose library, reads the farm profile with the farmer's own token, fetches forecasts, identifies photos and creates speech. Every response uses the shape {success, data, message}.
Data and external services	Firebase Authentication, Cloud Firestore with Security Rules, Open-Meteo, Google Gemini, Azure Speech, YouTube, OpenStreetMap	Stores accounts and data, enforces who may read or write each document, and supplies weather, image analysis, speech, video and map tiles.

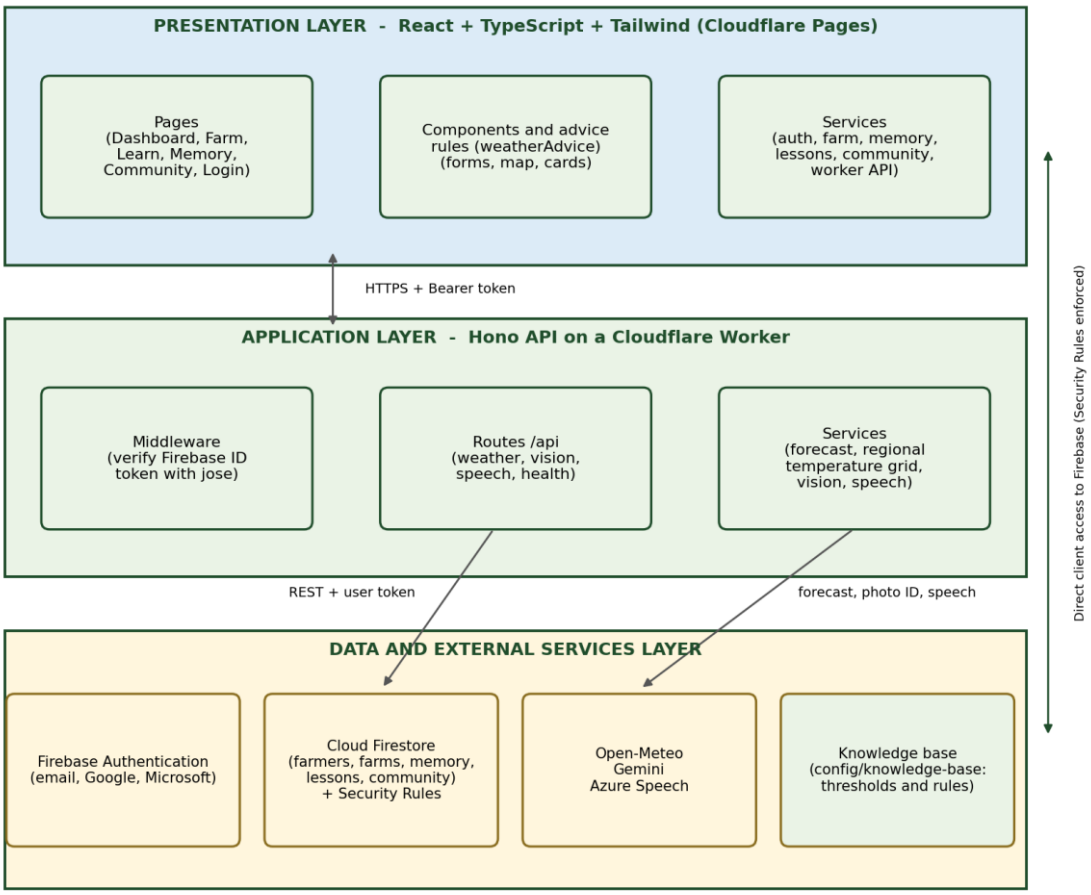


Figure 2: Three-layer architecture

Key design decisions:

- The client reads and writes Firestore directly, so Security Rules are the real permission check, not client-side code.
- The Worker uses the farmer's own ID token when it reads the farm profile, so the same rules still apply.
- Thresholds (for example heat at 35 °C or more, frost at 2 °C or less) are stored in the knowledge-base file and not inside components.

2.3 System design models

2.3.1 Use case diagram

The use cases below extend the Phase 1 diagram with photo identification (UC11) and plot measurement (UC12), which were added during development.

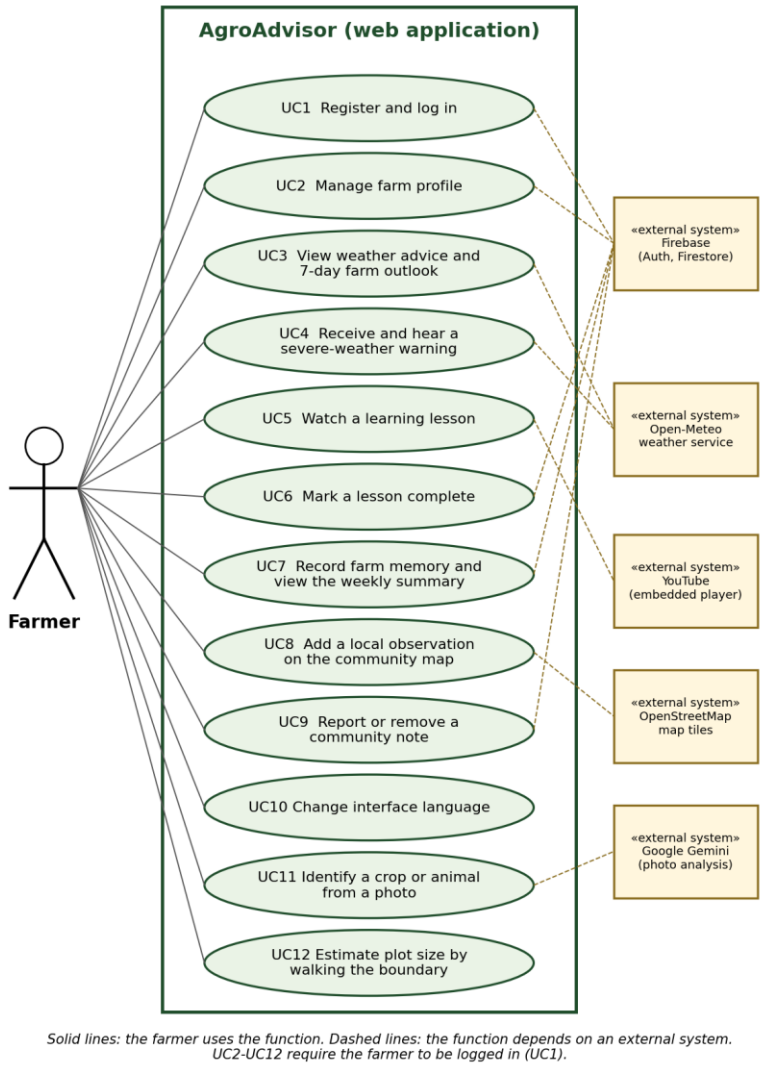


Figure 3: Use case diagram

Use case	Description	Main outcome
UC1 Register and log in	Farmer creates an account or signs in with email or Google.	Farmer reaches the dashboard.
UC2 Manage farm profile	Farmer enters or edits farm name, location, size, crops and livestock.	Advice is based on the saved profile.
UC3 View weather advice and 7-day outlook	Farmer sees current weather and a daily outlook with risk notes.	Farmer can plan the week.
UC4 Receive and hear a severe-weather warning	System shows a warning with action, timing and reason, and can read it aloud.	Farmer can protect crops and animals.
UC5 Watch a learning lesson	Farmer opens a reviewed lesson and plays the video.	Farmer learns a farm skill.
UC6 Mark a lesson complete	Farmer confirms the lesson is done.	A lesson entry is added to farm memory.
UC7 Record farm memory and view weekly summary	Farmer saves a weather, planting or success note.	Weekly counts and recent entries are updated.
UC8 Add a local observation	Farmer places a pin within 25 km of the farm with a short note.	Neighbours see the note on the map.
UC9 Report or remove a	Farmer reports another farmer's note, or removes	Reported notes are hidden

Use case	Description	Main outcome
community note	their own.	after two different reports.
UC10 Change interface language	Farmer selects one of five languages.	Screens appear in that language.
UC11 Identify a crop or animal from a photo	Farmer takes a photo and the system suggests a name.	The item can be added to the farm profile.
UC12 Estimate plot size	Farmer walks the boundary with GPS on.	Farm size field is filled in.

2.3.2 Activity diagram

The activity diagram shows the main flow of the system: from opening the dashboard to seeing advice or a severe-weather warning.

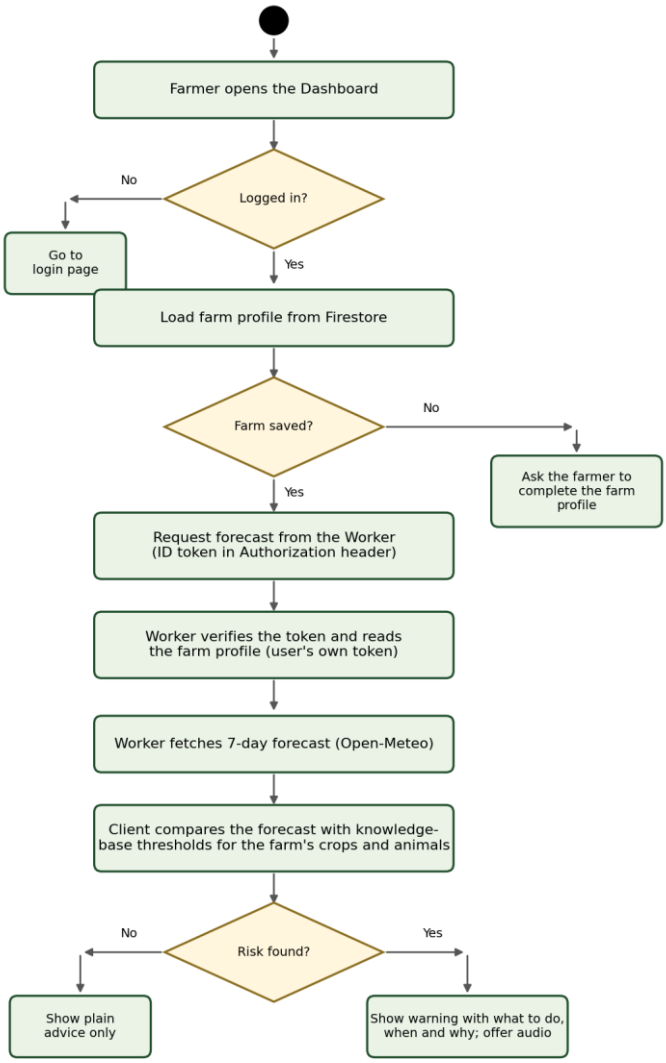


Figure 4: Activity diagram: view weather advice and warnings

2.3.3 State diagram

A community note moves through a small set of states. The rules allow a visible note to be reported only by another farmer and only once per farmer.

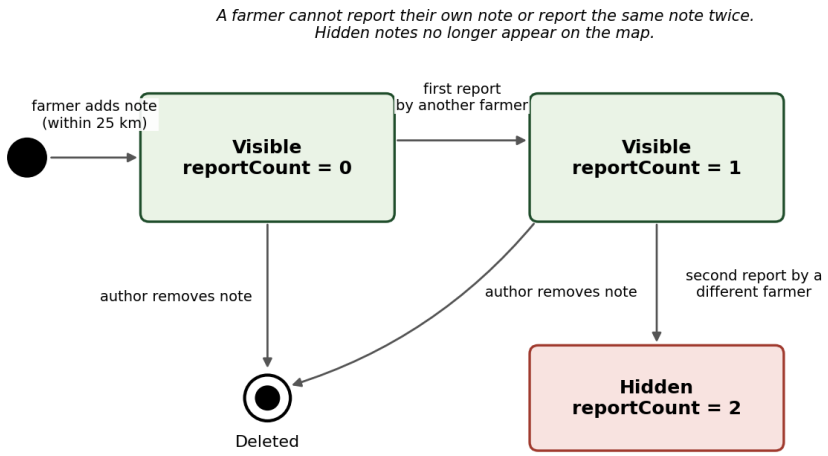


Figure 5: State diagram of a community note

2.3.4 Class diagram

The classes mirror the data stored in Firestore. The farmer and the farm share the same identifier, so a farmer has exactly one farm.

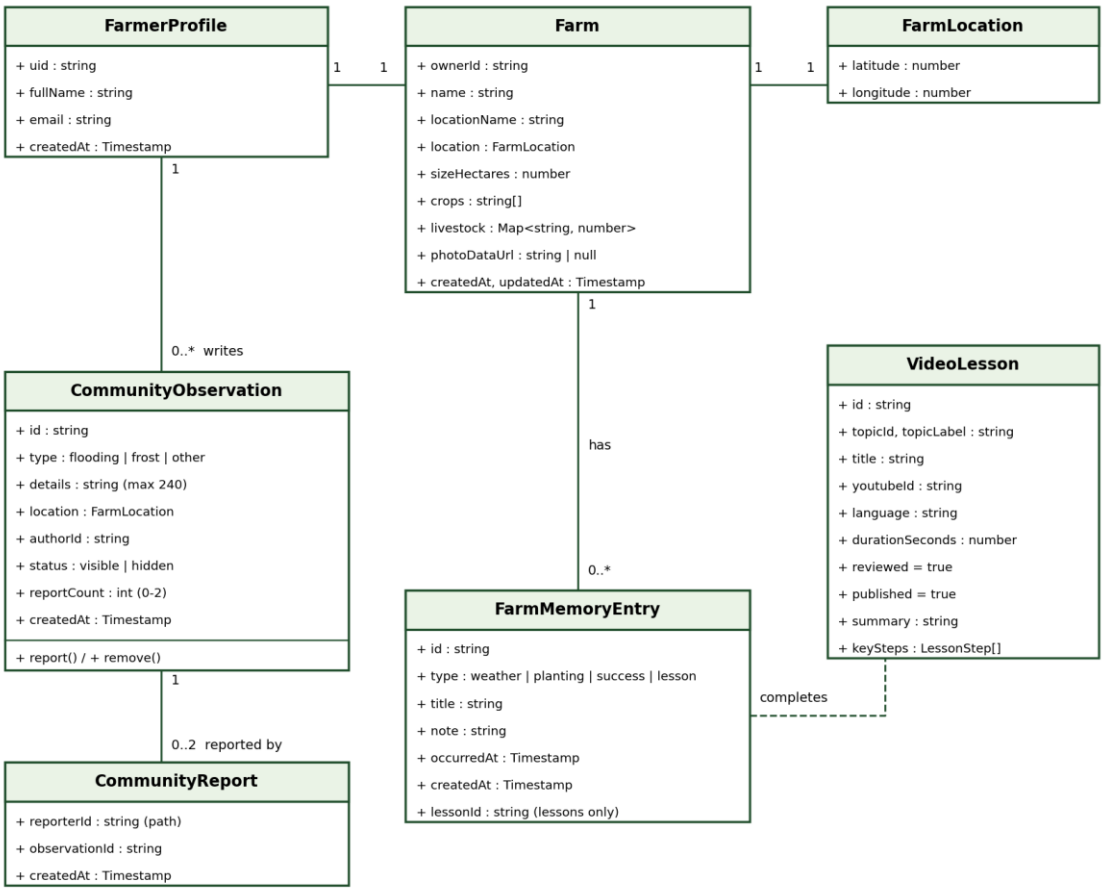


Figure 6: Class diagram

2.3.5 Sequence diagram

The sequence diagram shows how a forecast request travels through the client, the Worker, Firestore and Open-Meteo.

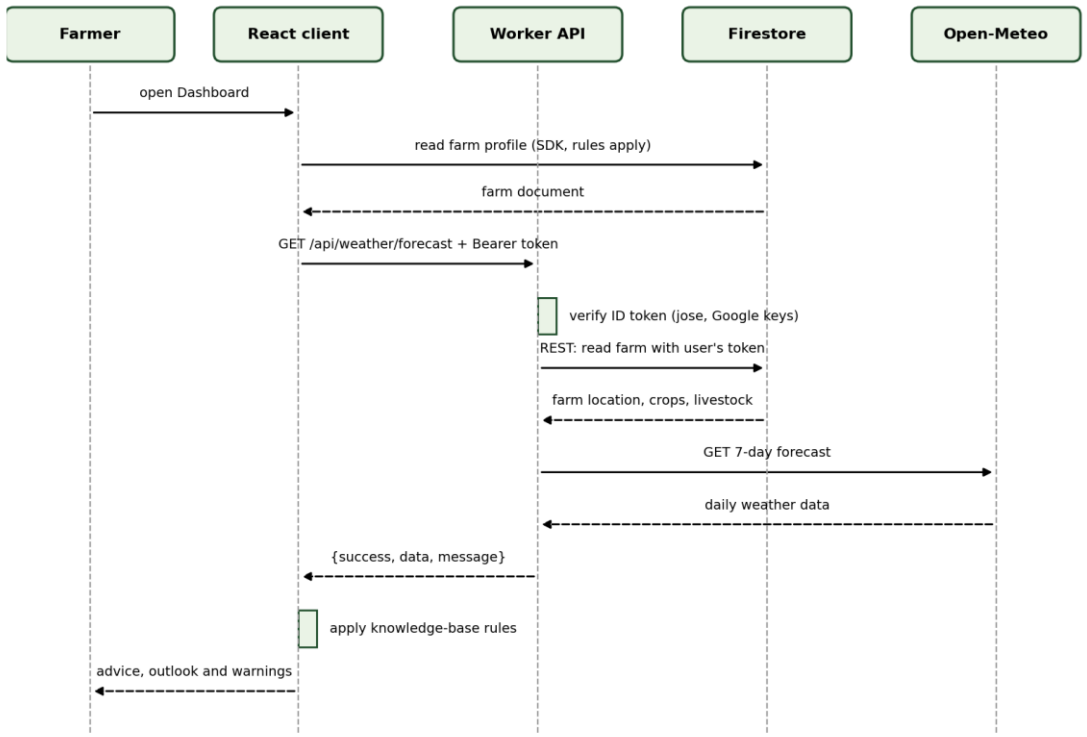


Figure 7: Sequence diagram: load farm weather advice

3. Project Management

3.1 Work breakdown

Each task lists its deliverables, milestone, resources, dependencies and constraints, and risks with contingencies. The durations are planned effort and must be replaced with the real dates of the project.

T1 Requirements and interview plan

Item	Detail
Deliverables	Interview questionnaire; list of problems; requirements list
Milestone	Phase 1 document approved
Resources needed	Student, supervisor, farmers to interview
Dependencies and constraints	None. Constraint: access to real farmers
Risks and contingencies	Risk: no farmer available. Contingency: use published sources and mark findings as unverified

T2 Design (UI, data, architecture)

Item	Detail
Deliverables	Context, architecture, use case, class, sequence, state and activity models; data design; colour theme
Milestone	Phase 2 document approved
Resources needed	Diagram tools, Figma or paper sketches
Dependencies and constraints	Depends on T1
Risks and contingencies	Risk: design changes during build. Contingency: keep the design modular and update models afterwards

T3 Authentication and farmer profile

Item	Detail
Deliverables	Register and login pages; Firebase project; farmer and farm services; farm profile form
Milestone	A farmer can register, log in and save a farm
Resources needed	Firebase Spark plan, Google sign-in setup
Dependencies and constraints	Depends on T2. Constraint: authorised domains must include the dev address
Risks and contingencies	Risk: login blocked by domain settings. Contingency: add the domain in Firebase and test email login first

T4 Weather advice and alerts

Item	Detail
Deliverables	Cloudflare Worker with forecast route; weather-advice rules; dashboard; severe-weather warning; knowledge-base file
Milestone	Dashboard shows advice and a warning for a real farm
Resources needed	Open-Meteo (free, no key), Wrangler, Hono
Dependencies and constraints	Depends on T3
Risks and contingencies	Risk: weather service outage. Contingency: clear error message and retry button

T5 Learn page and farm memory

Item	Detail
Deliverables	Lesson data and page; lesson completion; farm memory page with weekly summary
Milestone	A farmer can watch and complete a lesson and see it in memory
Resources needed	Reviewed video list, Firestore
Dependencies and constraints	Depends on T3
Risks and contingencies	Risk: unreviewed videos. Contingency: only show lessons marked reviewed and published; review videos before final submission

T6 Community map and security rules

Item	Detail

Item	Detail
Deliverables	Community map page; add, report and remove notes; Firestore Security Rules and indexes; rule tests
Milestone	Six rule tests pass and notes can be added and reported
Resources needed	react-leaflet, OpenStreetMap, Firebase emulator, Java
Dependencies and constraints	Depends on T3 (farm location)
Risks and contingencies	Risk: query needs an index that takes minutes to build. Contingency: deploy indexes early

T7 Photo identification and speech

Item	Detail
Deliverables	Vision and speech Worker routes; photo scan in the farm form; spoken warnings
Milestone	A photo returns a crop or animal name; a warning can be played as audio
Resources needed	Google Gemini and Azure Speech keys held as Wrangler secrets
Dependencies and constraints	Depends on T4
Risks and contingencies	Risk: keys missing or quota used. Contingency: typed name entry and on-screen text still work

T8 Testing and fixes

Item	Detail
Deliverables	Type checks, build, rule tests, manual browser tests of each page; fixes
Milestone	All pages work with a real account and sample data
Resources needed	Browser, test accounts, seed data
Dependencies and constraints	Depends on T3 to T7
Risks and contingencies	Risk: late bugs. Contingency: test each module when it is finished

T9 Documentation

Item	Detail
Deliverables	Phase 1, Phase 2, Phase 3 and final report
Milestone	Each document handed in on time
Resources needed	Word, diagram scripts, screenshots
Dependencies and constraints	Runs alongside T1 to T8
Risks and contingencies	Risk: documents fall behind the system. Contingency: update documents at each milestone

T10 Deployment and final review

Item	Detail
Deliverables	Client on Cloudflare Pages; Worker deployed; final demonstration
Milestone	System live and demonstrated

Item	Detail
Resources needed	Cloudflare account, Wrangler, Firebase deploy
Dependencies and constraints	Depends on T8
Risks and contingencies	Risk: environment differences. Contingency: test the deployed build with a fresh account

3.2 Gantt chart

The chart shows the planned order and overlap of the tasks in project weeks. [Replace the week numbers with calendar dates agreed with the supervisor.]

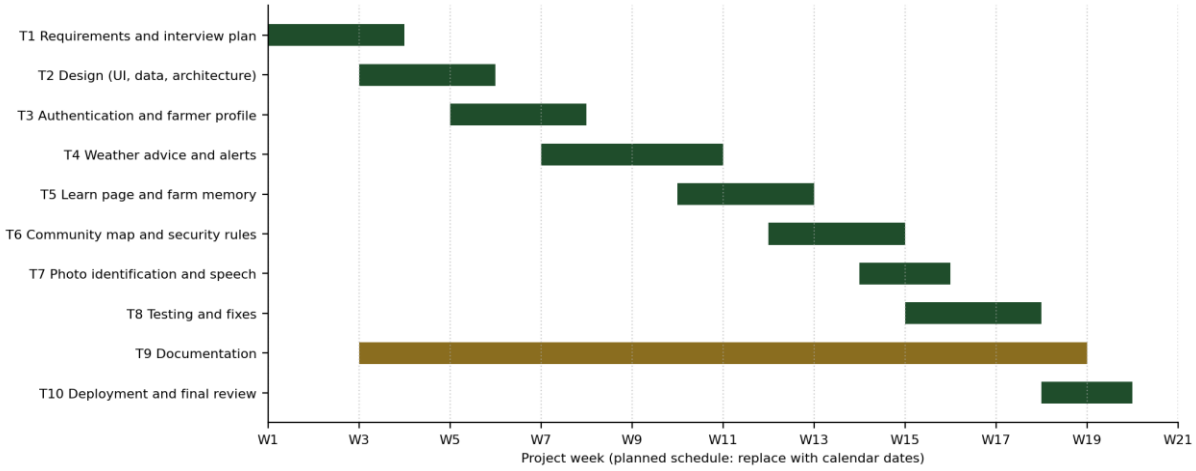


Figure 8: Gantt chart (planned schedule)

4. Conclusion

This document defines what AgroAdvisor must do and how it is built. The functional and non-functional requirements, the models and the work plan describe one system: a farmer-only advisory platform that turns weather data and the farm profile into advice and warnings. The one planned feature that is not yet built is background push notification (FR17).